

Predicting Real Estate Prices in the United States: A Data-Driven Approach

Maanya Bagga (23-451-164)
Vishal Varma Alluri (19-876-093)
Rakesh Varma Addada (19-876-135)

2024-12-02

Contents

Introduction	2
About the Dataset	2
Data Sampling	6
Data Exploration and Visualisation	6
Models	7
Model: Linear Regression	7
Model: XGBoost	9
Model: Lasso Regression	11
Model: Random Forest	12
Evaluation	14
Appendix	14

Introduction

The real estate market is inherently dynamic and prone to fluctuations making it a critical area of focus for diverse range of stakeholders. These include investors looking to maximize returns, employees working within the sector, tenants, property management teams maintaining assets etc. For these groups, access to reliable property price estimates is essential. This project aims to address this need by developing a robust predictive model using machine learning techniques. The application of machine learning in the real estate sector offers several advantages over traditional methods, like Comparable Market Analysis (CMA), which are often limited to small-scale predictive tasks. In the following section we discuss some of the advantages of using machine learning techniques to predict real estate prices.

Relevance of machine learning models in real estate sector

- Ability to handle large dataset - While traditional methods like CMA are effective for localized and small-scale price predictions, they struggle to process and analyze the vast volumes of data generated in the real estate sector. Machine learning models excel at handling these large datasets efficiently.
- Modeling Complex Relationships - Real estate prices are influenced by numerous factors, including location, property features, market trends, and economic conditions. Machine learning techniques are highly effective at modeling complex, non-linear interactions among these variables, which traditional methods often fail to capture.
- Improved Accuracy and Insights - Machine learning models can provide more accurate predictions and deeper insights by continuously learning from new data.

In this project, five different machine learning models were trained to predict real estate prices based on a predefined set of features like house size, number of baths and beds, land size etc. Each model was evaluated using standardized error metrics, including Root Mean Square Error (RMSE), Mean Absolute Error(MAE), Mean Squared Error(MSE) and R-squared to ensure a comprehensive assessment of their predictive performance. The metrics of each model were compared with their respective industry benchmarks to assess the performance of a each model.

About the Dataset

Source and Features The original dataset was sourced from kaggle. It was a csv file consisting of 2,226,382 observations representing real estate listings in the United States. The data is categorized by various attributes, including state, city, and ZIP code. The dataset had 12 features in total out of which 6 variables were categorical columns - **brokered by**, **status**, **street**, **city**, **state** and **zip_code**. Brokered by and street were numerically encoded for privacy purposes. The target variable **Price** represents the sale price of a property in dollars. The variable **house_size** is a numeric variable representing the living space (i.e. area in which the house is constructed) while **acre_lot** refers to the total land area of the property. In case the property has been sold previously, the column **previously sold date** includes a date, otherwise it is null. The data was originally parsed from Realtor.com, a prominent real estate listing website in the USA. There is no indication whether a particular listed property is for commercial or private use.

Data Cleaning Before Sampling

1. Handling Missing Values

The graph below visually represents the distribution of NA values across all the variables. The variable house size has the highest proportion of NA values followed by bath and bed. Additionally, approximately 0.069% values in the target variable, Price, were missing, necessitating the removal of those observations.

To address missing data, all rows with missing values (NA) were dropped, reducing the dataset size to around one million observations. Row-wise elimination was preferred over column-wise elimination because the features with the highest proportion of missing values were crucial to the analysis. Excluding these features would have significantly diminished the predictive power of the model.

2. Adjusting the State Variable

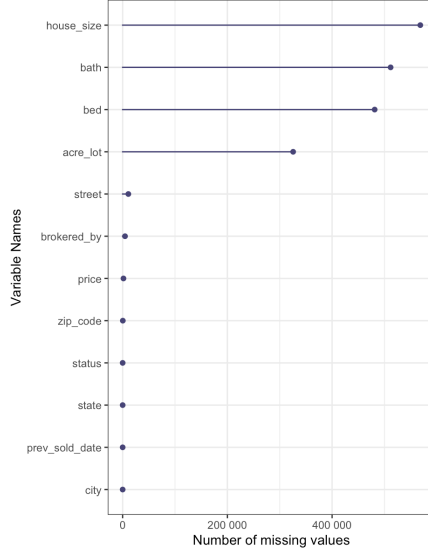


Figure 1: Distribution of NAs across Variables

The original dataset contained 53 unique values for the state variable, representing all U.S. states, including territories like Puerto Rico and Guam. However, for the purpose of this analysis, only the mainland United States (i.e., the 50 states and the District of Columbia) were considered relevant. As a result, the states and territories outside the mainland U.S., such as Puerto Rico and Guam, were excluded from the dataset. This adjustment reduced the state variable to 51 unique values, including the District of Columbia.

3. Duplicate Identification and Removal

The original dataset lacked a unique identifier for each record, which made it challenging to detect and handle duplicate entries. To address this, a composite key was created by combining multiple variables that, together, uniquely identified each listing. Specifically, the composite key was formed by concatenating the following variables: zip_code, street, city, house_size, bed, bath, acre_lot, brokered_by, status, prev_sold_date, price, and state.

This combination of attributes was chosen because it captures key characteristics that, when taken together, should be unique for each property listing. By using this composite key, we ensured that no perfect duplicates existed within the dataset. Any entries that had identical values across all these variables were considered duplicates and were removed, ensuring that each record in the dataset was unique.

However, in order to perform the necessary aggregations for our analysis, certain variables were deemed unsuitable or irrelevant for inclusion in the composite key. Specifically, the brokered_by variable was initially part of the composite key, but upon reviewing the data, it became clear that its inclusion resulted in a maximum sales frequency of 3. Without the broker variable, the maximum sales frequency increased to 4. Given that the goal of this analysis is to understand the overall property sales trends, including the broker in the composite key introduced unnecessary granularity. Therefore, variables including brokered_by, status, prev_sold_date, price, state were excluded from the composite key after duplicate removal. This adjustment streamlined the data.

4. Creating a New Variable: Sales Frequency

To enhance the analysis, a new variable, sales_frequency, was created to capture the number of times each property had appeared in the dataset. This variable allowed for the identification of properties that had been sold multiple times over the course of the dataset. To achieve this, we first aggregated the data by property id and counted the number of occurrences of each house in the dataset, thereby determining how frequently each property had been listed and sold. For properties with multiple sales records, we focused on retaining only the most recent transaction. This was accomplished by sorting the data by the previous sale

date and selecting the entry with the latest sale date for each unique property. The process of sorting and selecting the most recent sale ensured that only the most up-to-date information was kept, and any previous sale records were excluded. This step was crucial for analyzing current property sales trends, as outdated sale records might have skewed the results. Subsequently, the column previously sold date, which indicated earlier sales dates, was removed from the dataset. This was done because, after selecting the most recent sale for each property, the previous sale dates no longer provided additional useful information and were redundant.

5. Outlier Detection and Removal

House Size- In the dataset, it was observed that 75% of the houses had a size below 2,484 square feet, while the maximum value appeared unusually large at 1,560,780 square feet. This outlier suggests that there may be extreme values that do not accurately reflect the typical distribution of house sizes in the dataset. These could have been extremely luxurious house that do not represent the general population or a data entry error. To address this, we decided to perform outlier detection using the Interquartile Range (IQR) method. The calculated lower bound was found to be negative. Since house sizes cannot be negative, this result indicates that no house size values needed to be eliminated on the lower end of the distribution. The upper bound, was calculated as 4167 square feet. Based on the IQR method, any house size larger than this value was considered an outlier and was removed from the dataset.

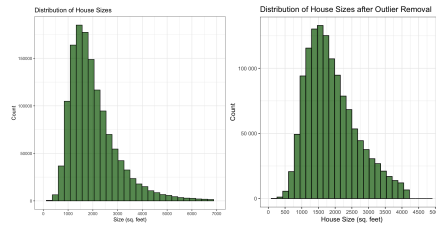


Figure 2: Before vs After Outlier Removal

Beds - An initial exploration of the bedroom data revealed that 75% of houses have up to 4 bedrooms. However, there is an extreme value of 444 bedrooms, which is highly unusual and likely an outlier. To further identify potential outliers, the interquartile range method was employed. The interquartile range for bedrooms shows that 50% of properties have number of bedrooms within a range of 1. Any properties with bedrooms below 1.5 and above 5.5 have been eliminated.

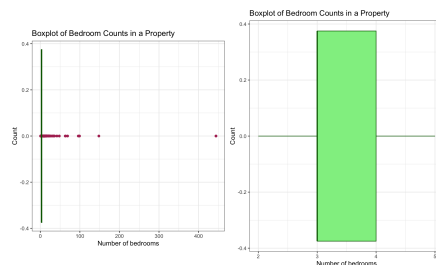


Figure 3: Before vs After Outlier Removal

Baths - An analysis of the bath variable revealed that 75% of the properties have up to 3 bathrooms, however, there are some properties with 175 bathrooms. This number would either be a potential data entry or an outlier. To manage the outlier, Interquartile range method was applied. The lower bound was 0.5, which is less than the minimum observed value of baths in the dataset, therefore no cuts were made at the lower end of the tail. The upper bound of 4.5 suggests that properties with more than 4.5 bathrooms can be potential outliers. Therefore, we remove the properties with more than 4 bathrooms.

Acre_lot To ensure uniformity in the dataset, land sizes recorded in acres were converted to square feet. This allowed for a consistent unit of measurement across all measurement variables, facilitating meaningful

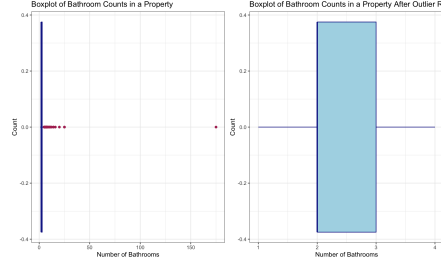


Figure 4: Before vs After Outlier Removal

comparisons and analysis. 75% of the properties have a land size of less than or equal to 18,300 square feet. The dataset includes an extreme maximum value of 4,356,000,000 square feet, which was not properly plotted due to its magnitude, as illustrated in the first image below. An interquartile range (IQR) analysis was performed to identify potential outliers. The calculated lower bound was less than the minimum value in the dataset, signifying no outliers on the lower end. However, it was observed that some entries had a land size of 0, even though the corresponding house size was positive. This scenario is logically inconsistent, as it is not feasible for a house to exist without any land. To address this issue, all observations meeting these criteria (land size greater than house size) were removed from the dataset.

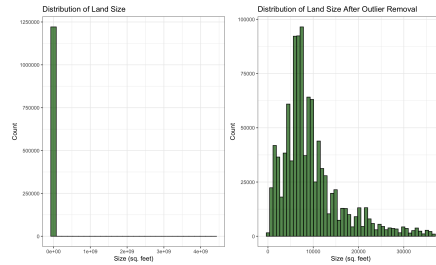


Figure 5: Before vs After Outlier Removal

Target Variable - Price To ensure the accuracy of the target variable, price, several inconsistencies were identified and addressed. There were houses priced between \$1 and \$5,000 in states like New York and California, which are unlikely to reflect realistic values and were flagged as potential data entry errors, necessitating their removal. Similarly, prices for small houses or studio apartments were unrealistically high in some states, particularly in California and New York. For instance, a 123 sq. ft. house in Texas was priced at \$160,000, while a 1,221 sq. ft. apartment in Alabama was listed at \$8,000, both of which are implausible. A detailed Price-to-House Size Ratio analysis revealed extreme outliers, such as properties priced at \$4,000 per sq. ft., likely indicating luxury homes. Minimum price-to-size ratios were also evaluated against typical residential market ranges for each state. Discrepancies were noted in states like California, where the minimum ratio of 37.1 was below the usual range of 100–200, and Delaware, where 18.7 was far below the expected range of 50–80. Additionally, states like Florida (18.1), Georgia (19.4), Michigan (1.89), Missouri (0.0006), and New York (5.79) displayed ratios too low for their respective housing markets, while Alaska's minimum ratio of 215 was unusually high. These anomalies, along with unrealistic price-to-size ratios in states such as Ohio (8.44), Pennsylvania (12.0), South Carolina (16.6), South Dakota (15.6), and Virginia (11.1), were systematically reviewed to refine the dataset and ensure reliable price data for subsequent analysis.

These preprocessing steps, dealing with missing values, duplicates, irrelevant regions, outliers, and transformations, were essential to refine the dataset and make it suitable for analysis and modeling. After the data cleaning process the data was ready to be sampled.

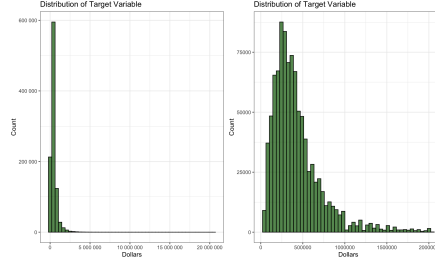


Figure 6: Before vs After Outlier Removal

Data Sampling

Stratified sampling was employed to sample the dataset, ensuring that the resulting sample is balanced and accurately represents the population. In this case, stratified sampling was applied based on three attributes: `house_size`, `bath`, and `beds`, ensuring that properties of all sizes were adequately represented. The sample contained 14,928 observations, which seemed like a sufficient size to train and evaluate machine learning models while also reducing computational costs associated with datasets containing high cardinal categorical variables.

Data Exploration and Visualisation

- **Distribution Analysis:** Visualized histograms for price, `house_size`, and `log_land_size` to understand data spread and outliers.
- **State Level Insights:** Identified top states for affordable and luxury properties, comparing prices per square foot and total counts.
- **Relationship Analysis:** Correlation matrix and scatter plots revealed strong positive relationships between price, `house_size`, and `bath`.

Practical Insights

Bathroom-to-Bedroom Ratio Analysis The bathroom-to-bedroom ratio was calculated to evaluate how many bathrooms are present per bedroom in each property. Generally, having up to two additional bathrooms than the number of bedrooms is considered acceptable for most real estate standards. The analysis revealed that the majority of properties adhere to this guideline, with a maximum of two bathrooms per bedroom recorded. This indicates that the dataset is within an acceptable range for bathroom-to-bedroom ratios, and no extreme or unrealistic values were identified that would require further exclusion.

Correlation Matrix This heatmap illustrates the correlation between numeric variables in the dataset, including price, number of bedrooms (`bed`), number of bathrooms (`bath`), house size, sale frequency, and land size. Strong positive correlations are observed between variables such as house size and number of bathrooms (0.71) and between house size and number of bedrooms (0.62). On the other hand, land size and sale frequency show minimal correlation with price and other variables, suggesting limited influence on these factors.

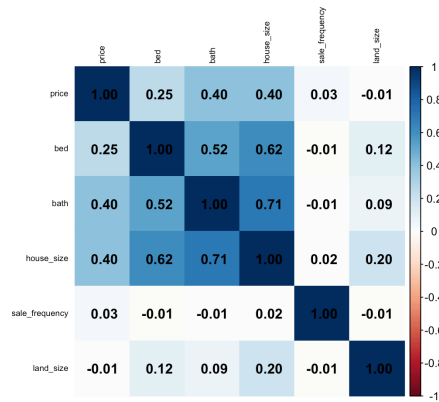


Figure 7: Correlation Matrix

Models

Model: Linear Regression

Linear regression was chosen as the baseline model for this project due to its simplicity and effectiveness, as evidenced by the literature. It is widely recognized in the real estate domain for its low prediction error when capturing relationships between features and target variable i.e. price. However, despite its merits, this model was not a perfect fit for the dataset at hand due to the potential issue of high cardinality among categorical variables, which if one hot encoded would lead to $p > n$ (variables > observations). To address this, categorical variables with low frequency were grouped into an “Others” category to reduce dimensionality and prepare the data for encoding.

One Hot Encoding Before building the model, one-hot encoding was applied to the categorical variables to ensure compatibility with the regression model while maintaining a manageable number of predictors. To address high cardinality, categorical variables with low frequencies were grouped into an “Others” category before encoding. The variables subjected to one-hot encoding included status, states, brokered_by, zip_code, and city.

Here are the statistics before and after grouping:

- Status: Initially, 7,000 unique values, reduced to 24 after grouping.
- States: Initially, 4,000 unique values, reduced to 15 after grouping.
- Brokered by: Initially, 13,816 unique values, reduced to 11 after grouping.
- Zip Code: Initially, 28,412 unique values, reduced to 14 after grouping.

While most variables were grouped before encoding, states and status were directly encoded. For states, an additional approach was tested: grouping them into four major regions (e.g., Northeast, South, Midwest, and West) before applying one-hot encoding to these broader categories. This method was evaluated to determine its impact on model performance.

Forward Selection Prior to performing feature selection using forward selection, the dataset was divided into training and test sets, following a 70-30 split. The unique id variable was excluded from the analysis, as it was deemed unnecessary for model construction.

For the forward selection process, we setup a variable called ‘regressors’, which includes all variables except for the dependent variable. The process started with a null model, consisting only of the intercept. The

Residual Sum of Squares (RSS) is initially computed for the intercept-only model. Subsequently, each remaining predictor is evaluated by calculating the RSS for candidate models that include one additional predictor at a time. The predictor that minimizes the RSS is selected, provided that its inclusion improves the model. Finally, the selected predictors are displayed as the final set of features for the model. As per the forward selection process, the best model initially included 100 regressors. However, half of these regressors were found to be statistically insignificant and were subsequently removed to reduce noise and minimize the risk of overfitting. After removing statistically insignificant variables, we were left with 34 regressors. Now we moved on to building and evaluating various linear models.

Model Variants Tested

In total six variations of the linear regression model were estimated to evaluate performance, namely:

- **Linear-linear:** This model explained 50.73% of the variation in the data, with a residual standard error (RSE) of 212,400.
- **Log-linear:** - In this model, the dependent variable (price) was log-transformed. Compared to the linear-linear model, the range of residuals narrowed, and the median residual was 0.0043, indicating a balanced distribution of errors. The adjusted R-squared improved to 56.89%, and the RSE suggests a deviation of approximately 0.4426 log units from the actual price.
- **Log-log:-** This model showed further improvement, with an RSE of 0.441 log units and an adjusted R-squared of 57.2%. Interestingly, `land_size` was found to be statistically insignificant and did not improve the model, so it was removed.
- **Multiple linear Regression:** This model included only the most impactful predictors—`house_size`, `baths`, `beds`, and `state_California`. However, its performance lagged behind the other models, with an RSE of 0.4876 log units and an adjusted R-squared of 45.5%.
- **Interaction between house size and land size:-** While this model produced an RSE of 0.4426 log units and an adjusted R-squared of 56.89%, the interaction term was not statistically significant. Thus, there is no evidence to suggest that including this interaction term improves the model.
- **log-log with regional dummies:** - In this model, states were grouped into regions, and the state variable was replaced with three regional dummy variables. While the model achieved an RSE of 0.4648 log units and an adjusted R-squared of 52%, it underperformed compared to the original log-log model. Interestingly, the coefficients for the regional dummies were negative, in contrast to the uniformly positive coefficients observed for state dummies in the previous model.

Out of the above listed models the first three (Linear-linear, Log-linear, and Log-log) were estimated under two different transformations of the data—random sampling & stratified sampling. Among these, the log-log model demonstrated the best performance under both data. However, an analysis of the residual plots under random sampling revealed that the residuals were not randomly distributed. This issue was addressed when stratified sampling technique was applied, the residuals now seemed to have a roughly constant variance. Therefore, the model with stratified was chosen as the best one, even though the R-squared value was slightly higher with the random sampling (60% with random sampling vs 57.2% with stratified). An analysis of the Q-Q plots showed the residuals closely followed the dashed line except the tails on either side which exhibited notable deviations, suggesting the presence of extreme outliers, therefore model is expected to be impacted in case of predictions related to extreme values (Luxury houses).

Considering the three charts below, it can be deduced that the Log-Log Model seems to have a better relationship for a linear model, as the log transformation improves the linearity and reduces heteroscedasticity. Therefore, the log-log model was chosen as the final model.

$$\text{Log}(\text{price}_i) = \text{Log}(\text{HouseSize}_i) + \text{FinalRegressors}_i + u_i$$

However, it failed the Breusch-Pagan test, indicating Heteroskedasticity in the model residuals. Therefore, we used robust standard errors.

Log-Log Model Performance Metrics

The performance of the log-log linear regression model was evaluated using standard metrics as represented below -

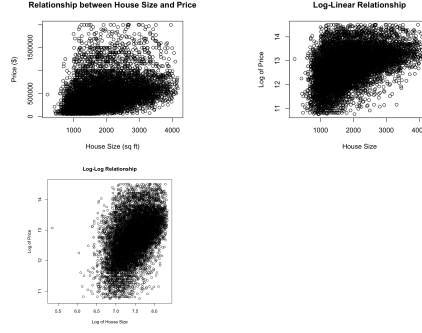


Figure 8: Relationship between House Size and Price

Metric	Value
Mean Absolute Error (MAE)	136,204.80
Mean Squared Error (MSE)	50,681,092,062
Root Mean Squared Error (RMSE)	225,124.60
R-Squared	48.48%

Results

RMSE: On average, the model exhibits a deviation of \$225,124.60 per prediction. The RMSE value is reasonable given that the price range of the properties in the dataset range from hundred thousand to millions.

MSE: The high MSE indicates that outliers have a significant impact on the results. This suggests that the variance in the predicted values increases for higher-priced properties.

R-Squared: The model explains 48.48% variability seen in the target value.

The model demonstrated a notable improvement after adjusting the outlier thresholds for various variables.

Comparison to Benchmark Values

Studies in the real estate domain indicate that values of R-squared typically range between 0.40 and 0.80. Simpler models applied to high-variance datasets often yield values in the lower range. For real estate models in expensive markets such as New York City or San Francisco, RMSE greater than \$100,000 are common due to high property values. In light of these benchmarks, the performance of the baseline linear regression model aligns with expectations for real estate price prediction ,especially when dealing with datasets characterized by substantial variability.

Model: XGBoost

XG Boost or Xtreme Gradient boosting is an ensemble learning technique i.e. it combines the predictions of multiple individual model to build a more accurate predictive model. It's ability to handle complex relationships in data is the reason, it was chosen to model real estate price predictions. The development of the XGBoost model for predicting real estate prices involved several steps as mentioned below:

Data Preprocessing: All categorical variables in the dataset were converted into factors. These were then label encoded, which is crucial for transforming categorical data into a numerical format that can be understood by the XGBoost algorithm.

Dataset Splitting: The dataset was randomly split into training and test sets, with 70% of the data used for training the model and the remaining 30% reserved for testing its performance.

Feature and Target Separation: The target variable (prices) and the features were separated to ensure the model only learned from the relevant features for price prediction.

Hyperparameter Tuning: To further improve the model performance, a hyperparameter tuning process was carried out. Two distinct methods for hyperparameter tuning were applied:

Hyperparameter Tuning with Grid Search and Cross-Validation In this analysis, we aimed to optimize the performance of the XGBoost model by tuning key hyperparameters using grid search and cross-validation. The following steps outline the process:

Hyperparameters: A grid of hyperparameters was defined for optimization. The selected hyperparameters included:

- `max_depth`: The maximum depth of a tree (values considered: 3, 5, 7).
- `eta`: The learning rate (values considered: 0.01, 0.1, 0.2).
- `subsample`: The fraction of samples to be used for training (values considered: 0.6, 0.8, 1.0).
- `colsample_bytree`: The fraction of features to be used for each tree (values considered: 0.6, 0.8, 1.0).

Cross-Validation A 5-fold cross-validation was performed for each combination of hyperparameters. For each iteration, the model was trained using the training data (`dtrain`), and the root mean squared error (RMSE) was used as the evaluation metric.

Parameter Selection: The code looped over each combination of hyperparameters in the grid. For each combination, the model was trained and evaluated using cross-validation. The RMSE was tracked for each combination, and the hyperparameters yielding the lowest RMSE were selected as the best set. The best parameters were learning rate of 0.1, maximum depth of 7, subsample of 0.8 and col sample of 0.6. The RMSE was 169201.80

Hyperparameter Tuning with Random Search and Cross-Validation A random search was employed to explore a broader range of hyperparameters. Random search involves sampling random combinations of hyperparameters from predefined ranges, rather than exhaustively testing all possible combinations (as in grid search). The following steps outline the random search and cross-validation process:

Random Sampling: A set of 10 random samples was generated for each hyperparameter of interest:

- `max_depth`: Random integers between 3 and 7.
- `eta`: Random values between 0.01 and 0.3.
- `subsample`: Random values between 0.6 and 1.0.
- `colsample_bytree`: Random values between 0.6 and 1.0.

Cross-Validation: For each set of randomly sampled hyperparameters, 5-fold cross-validation was performed.

Parameter Selection: The best performing combination of hyperparameters was `eta` of 0.2721691, `max_depth` of 4, `subsample` of 0.8950382, `colsample_bytree` of 0.7518237 and the corresponding RMSE value was 172329.10

Final Model: - Grid Search yielded a lower RMSE compared to Random Search, indicating that it provided a better set of hyperparameters for the XGBoost model. Therefore, the parameters chosen by Grid search were used for training our model.

The performance of the final XGBoost model is represented below -

Metric	Value
Mean Absolute Error (MAE)	103769.60
Mean Squared Error (MSE)	26,226,219,981
Root Mean Squared Error (RMSE)	161945.10
R-Squared	70.28%

Results

RMSE: On average, the model exhibits a deviation of \$161945.10 per prediction which is an improvement as compared to linear regression.

R-Squared: The model explains 70.28% variability seen in the target value.

Comparison to Benchmark Values Typical RMSE values for XGBoost in real estate datasets can range from \$100,000 to over \$200,000 depending on data scale for instance in high-value markets (e.g., luxury properties in major cities). In practice, R^2 values in real estate models often range from 0.6 to 0.8, with values above 0.7 are considered good.

Model: Lasso Regression

Lasso regression is a linear regression method with regularization that shrinks less important feature coefficients to zero, effectively performing feature selection. It helps reduce multicollinearity, prevent overfitting, and improves model interpretability. In this project, Lasso is useful for identifying key factors like house_size and bath while excluding irrelevant ones. This ensures better predictions and a simpler, more robust model.

Implementation of Lasso Regression.

The implementation involved the following steps: **Data Preprocessing:** - We applied log transformations to key numerical variables like price, house size, and land size. This reduced skewness and stabilized variance, ensuring linear relationships between variables. - We scaled the numerical features to standardize their values, which is crucial for regularized regression techniques like Lasso. One-hot encoding was applied to categorical variables, such as state, to convert them into numerical format. **Feature Engineering:** Several new features were created to capture complex relationships in the data: **Price per square foot:** An indicator of property value relative to size. **Bed-to-bath ratio:** Captures balance between bedrooms and bathrooms. **Land-to-house size ratio:** Reflects the proportion of land to house size. **Bed-bath interaction:** Highlights the combined effect of these features. **Total rooms:** Sum of bedrooms and bathrooms, indicating house capacity. **Room-size interaction:** Captures the relationship between total rooms and house size. **Outlier Removal:** Extreme outliers in price and price_per_sqft were removed using the interquartile range (IQR) method. This helped reduce the influence of extreme values on the model. **Splitting Data:** The dataset was split into 80% training data and 20% testing data to evaluate the model's performance on unseen data. **Cross-Validation:** We used 10-fold cross-validation to identify the best regularization parameter (λ). The cv.glmnet() function automatically split the training data into 10 subsets. The model was trained on 9 subsets and validated on the 10th, repeating the process for all subsets. The best λ was selected based on the lowest mean squared error (MSE). **Model Training:** Lasso regression was trained using the features and target variable (log_price). After training, irrelevant or less impactful features had their coefficients shrunk to zero. ### Model Evaluation

After training the Lasso regression model, its performance was evaluated on the test dataset. Key metrics were computed to assess its predictive ability, as summarized below: | Metric | Value | |-----| |-----| | Root Mean Squared Error (RMSE) | 141,285 | | Mean Absolute Error (MAE) | 109,236 | | Mean Squared Error (MSE) | 19,961,324,872 | | R-Squared | 42.9% |

Interpretation:

MAE: On average, the model's predictions deviate by approximately \$109,236 from the actual property prices. **RMSE:** The average prediction error is \$141,285, with higher weights for larger errors. **R-Squared:** The model explains 42.92% of the variability in property prices, indicating moderate predictive power.

- The most significant predictor is `log_house_size`, indicating that house size plays a dominant role in determining property prices.
- Features like `log_land_size`, `bed`, `bath`, and `bed_bath_interaction` were deemed non-contributory by Lasso, as their coefficients were shrunk to zero.
- `bed_bath_ratio` and `total_rooms` also play a role in influencing prices, though their effects are smaller compared to house size.
- The Lasso model has effectively removed irrelevant or redundant features, simplifying the model while retaining key predictors.

Model: Random Forest

The Random Forest algorithm was selected for its robustness to overfitting and ability to capture complex non-linear relationships, making it a highly suitable choice for predicting house prices. Additionally, it handles categorical variables as factors and does not require complex preprocessing. It handles multi-collinearity well and outliers have a very limited impact due to the averaging effect of multiple decision trees.

Data Preprocessing

Region-Based Encoding States were grouped into four regions (Northeast, Midwest, West, and South) to reduce dimensionality while retaining geographical context. Regions and the status variable were one-hot encoded to effectively capture their influence on house prices.

Frequency Encoding for High-Cardinality Features The city variable was frequency-encoded to address its high cardinality, ensuring that its statistical relevance was incorporated without causing dimensionality issues. The city column was later removed after encoding to avoid redundancy.

No further interaction terms were created as Random Forests are capable of analysing non-linear relationships.

Robust Feature Selection: The final chosen variables were - Target variable: price - Continuous variables: `house_size`, `land_size`, `bed`, `bath`, `sale_frequency`, - Encoded categorical variables: region (states) and status (sold or unsold), and `city_freq`. Features such as `street`, `state`, `brokered_by`, `zip_code`, and others irrelevant or too granular for predictive modeling were excluded to prevent overfitting.

Sampling Strategy

A stratified sampling technique was used to split the data into training (80%) and testing (20%) sets. Stratification ensured that the price distribution in the training and testing sets closely matched the overall dataset which minimizes potential sampling bias.

Hyperparameter Tuning

To optimize the Random Forest model, a structured grid search with cross-validation was implemented:

- **Cross-validation:** A 10-fold cross-validation approach was used to ensure model stability.
- **Tuning Grid:** The grid search explored the following hyperparameter combinations:
 - Number of variables sampled at each split (`mtry`): 2, 4, 6
 - Minimum node size (`min.node.size`): 1, 5, 10
 - Splitting criterion (`splitrule`): Variance

This tuning process identified the optimal hyperparameters, resulting in the best performance on test data.

Model Evaluation

The model was evaluated on the test dataset

Performance Metrics

Metric	Value
Root Mean Squared Error (RMSE)	\$231932
Mean Absolute Error (MAE)	\$150,142
Mean Squared Error (MSE)	53,792,452,624
R-Squared	45.3%

Analysis - The model's average prediction error RMSE was \$231932 indicates a considerable average deviation of predicted prices from actual values. This is expected given the variability in house prices due to high-priced luxury properties. RMSE benchmark ranges from \$300,000 to \$400,000 so the model seems to perform well. - The MAE of \$150,142 indicates that, on average, the model's predictions deviate from the true price by approximately 43% of the median house price (\$350,000) again likely influenced by luxury properties, but this seems reasonable. - R_squared of 45.3% suggests that the model explains nearly half of the variance in house prices. This is moderate, it suggests that additional features may be required. —

Feature Importance

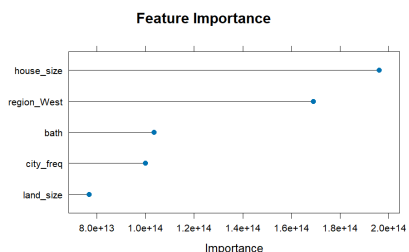


Figure 9: Feature Importance - RandomForest

The most influential features, based on impurity reduction, were: 1. `house_size` 2. `region_West` 3. `bath` 4. `city_freq` 5. `land_size`

This aligns with domain knowledge that house size, and geographic factors influence property prices.

Comparison of Variants Two Random Forest implementations were compared: 1. Using `ranger`: Faster training with cross-validation and target encoding resulted in robust performance. 2. Using `rf`: Provided similar performance metrics but required more computational time. Both implementations yielded comparable RMSE, MAE and R^2 , showing the model's consistency across frameworks.

Challenges and Limitations

- The high cardinality of the categorical variables increased the dimensionality of the dataset, resulting in higher computational costs. Moreover, with the condition where $p > n$ (the number of features exceeds the number of observations), additional challenges arose, as this often leads to overfitting and deteriorates model performance. To mitigate this, we grouped low-frequency variables before performing one-hot encoding. - Due to the high variation in the data, the standard errors were heteroscedastic. To address this, we computed robust standard errors to account for the unequal variability in the residuals. - In the initial implementation of random forest, the state and city variables were replaced with their respective average house prices (target encoding). While this approach tried to simplify the representation of geographic data, its reliability was questionable because it directly tied feature values to the target variable, this can lead to overfitting. - Another method involved assigning hierarchical labels to states based on their overall influence on house prices, this did not improve results. -The grid search process for tuning the XGBoost and RandomForest models took a considerable amount of time, highlighting the computational cost associated with optimizing multiple hyperparameters. - Overall, the model's performance could be improved by incorporating additional

domain-relevant features, such as proximity to amenities (e.g., schools, parks, or public transportation) or crime rates in the area. These variables are known to significantly influence house prices but were not available in the dataset.

Additional exploration of feature engineering and alternative sampling methods required.

Evaluation

Model	Mean Absolute Error (MAE)	Mean Squared Error (MSE)	Root Mean Squared Error (RMSE)	R-Squared
Linear Regression	136,204.90	50,681,092,062	225,124.60	48.48%
XGBoost	103,769.60	26,226,219,981	161,945.10	70.28%
Random Forest	150,142	53,792,452,624	231,932	45.3%
Lasso	109,236	19,961,324,872	141,285	42.92%

- XGBoost has the lowest MAE at \$103,769.60, indicating it makes the smallest average prediction error, closely followed by Lasso, while RandomForest has the highest. In terms of MAE, XGBoost consistently provides more accurate predictions compared to the other models.
- XGBoost also shows the highest R-Squared at 70.28%, indicating it explains the largest proportion of variance in house prices. Linear Regression achieves 48.48%, and Random Forest lags behind at 45.3%.
- Lasso performs well in terms of RMSE at \$141,285 followed closely by XGBoost at \$161,945.10, reflecting its superior handling of extreme price variations while Random Forest (\$231,932) performs the worst.

XGBoost outperforms the other models in all key metrics, with the lowest MAE and RMSE and the highest R^2 , making it the most accurate and reliable model for predicting house prices. Lasso Regression also shows competitive performance, particularly in managing large errors (MSE), while Random Forest and Linear Regression are less effective for this task in the given dataset. The lower performance of Random Forest could be due to insufficient hyperparameter tuning, overfitting on outliers, or its inability to fully capture the patterns.

Appendix

Extra and Preliminary visualizations

House Price Distribution The majority of properties fall within the 250K-500K category, followed by <250K. Higher price ranges, such as 750K-1M and >1M, have significantly fewer properties, indicating that lower to mid-range houses dominate the dataset which is in line with real world expectation. This data was later modified to account for outliers.

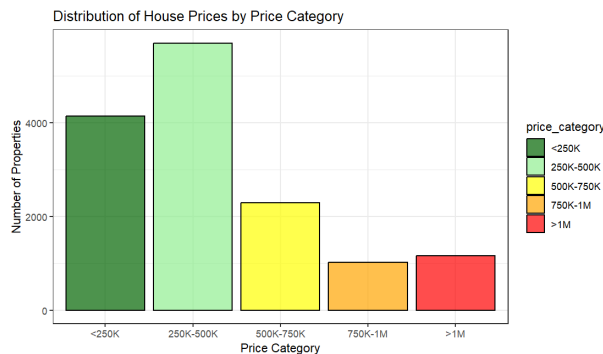


Figure 10: Distribution of Price

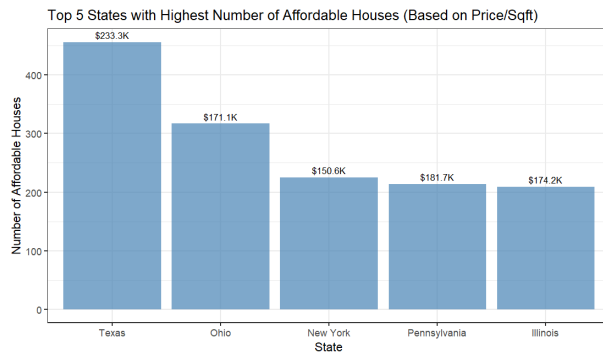


Figure 11: Top Affordable States

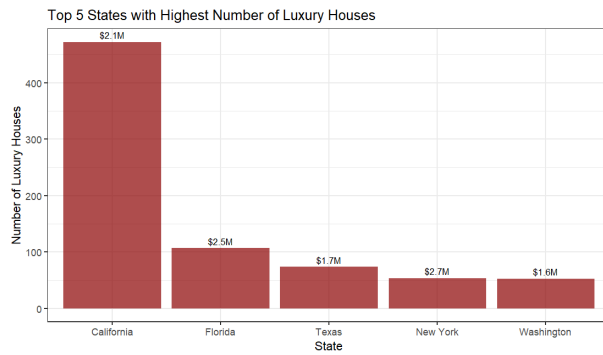


Figure 12: Top Luxury States.

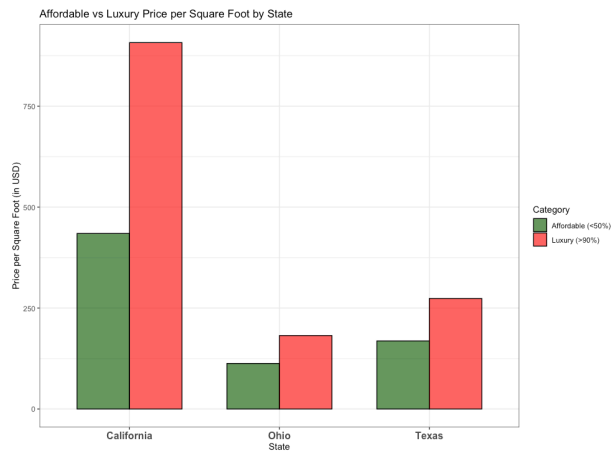


Figure 13: Affordable Vs Luxury Houses

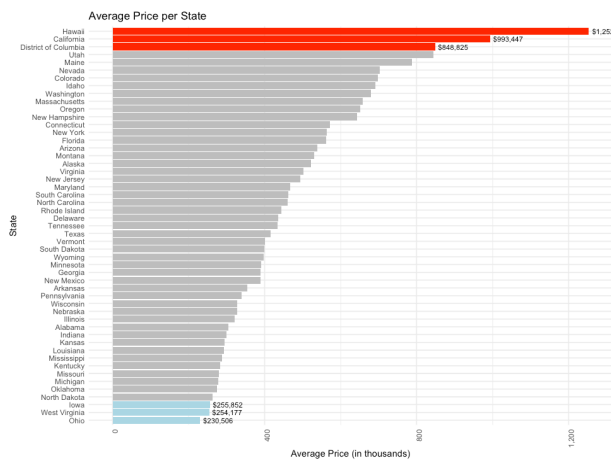


Figure 14: Average Price per state